

KNOWLEDGE CORPUS • ARCHITECTURE SPECIFICATION

Cloud-Native Security, Policy Enforcement & Observability

A Structured Technical Reference on Kubernetes RBAC, Secret Management, Prometheus Metrics, Grafana Dashboards, and OpenTelemetry Distributed Tracing

Doc ID: GL-CORPUS-06

Domain: Cloud Security & SRE

Topic: Security & Observability

Target: Graph RAG System

1. Prerequisites & Everyday Analogies

Before exploring security governance and observability pipelines, let us ground these concepts in an everyday analogy.

Imagine operating a modern international airport:

- **Security & RBAC (Role-Based Access Control):** Airport keycards and security checkpoints. A passenger ticket allows access to the boarding gate, but not the cockpit or baggage sorting rooms. Pilots and mechanics have specialized badges granting access only to areas required for their jobs.
- **Secret Management (AWS Secrets Manager):** The airport's central vault. Passwords, master keys, and encryption certificates are kept locked in a secure safe and handed out temporarily to staff with biometric authentication, rather than written on sticky notes left on terminal desks.
- **Metrics (Prometheus):** The digital dashboard in an airplane cockpit showing airspeed, fuel level, engine RPM, and altitude. It tracks quantitative numerical measurements over time.
- **Logs (Amazon CloudWatch / Fluentd):** The airplane's flight recorder (black box). It records a timestamped history of discrete events, messages, and warnings that occurred during the flight.
- **Distributed Tracing (OpenTelemetry):** The passenger luggage tracking system. When a bag travels through check-in, conveyor belts, security screening, cargo loading, and baggage claim, a single tracking barcode (Trace ID) tracks its journey across all intermediate systems.
- **Visualization & Alerting (Grafana & Alertmanager):** The air traffic control tower screens that visualize all flight paths and sound an emergency siren when an anomaly is detected.

2. Core Security & Observability Concepts

This section defines the foundational security and monitoring components used across modern cloud platforms.

Concept: Role-Based Access Control (RBAC)

Beginner-Friendly Definition: Role-Based Access Control (RBAC) is an authorization mechanism that regulates user and service access to computer or network resources based on assigned roles and privileges within an organization.

Why It Exists: To enforce the principle of least privilege, preventing unauthorized users or compromised microservices from accidentally or maliciously deleting critical cloud resources.

How It Works: Administrators create *Roles* defining allowed actions (e.g., `get`, `list`, `create`) on specific resources (e.g., pods, services). A *RoleBinding* binds these roles to specific users, groups, or automated ServiceAccounts.

Real-World Cloud Example: Granting a junior developer read-only access to view logs in the staging namespace while restricting production deployment permissions to senior CI/CD automation pipelines in Amazon EKS.

Related Concepts: Role, ClusterRole, RoleBinding, ClusterRoleBinding, ServiceAccount, Least Privilege.

Concept: Prometheus (Metrics Collection Engine)

Beginner-Friendly Definition: Prometheus is an open-source systems monitoring and alerting toolkit that collects, stores, and queries time-series numeric data (metrics) from running applications and infrastructure.

Why It Exists: Engineers need real-time numeric insights into system health (CPU utilization, memory consumption, request latency, and HTTP error rates) to detect system anomalies before outages occur.

How It Works: Prometheus operates on a *pull model*. It periodically sends HTTP GET requests (scrapes) to application endpoints exposing metrics in text format (e.g., `/metrics`). Metrics are stored as time-series data indexed by metric name and key-value label pairs, queried using PromQL (Prometheus Query Language).

Real-World Cloud Example: Scraping HTTP response codes and request durations every 15 seconds from an EKS microservice fleet and alerting the on-call engineer when 5xx errors exceed 2%.

Related Concepts: Time-Series Database (TSDB), PromQL, Pull Model, Scrape Target, Alertmanager, Exporter.

Concept: OpenTelemetry (Distributed Tracing Framework)

Beginner-Friendly Definition: OpenTelemetry (OTel) is a standardized, vendor-neutral observability framework providing APIs, SDKs, and tools to instrument, generate, collect, and export telemetry data (traces, metrics, and logs).

Why It Exists: In a microservice architecture where a single user click triggers calls across 10 different backend services, traditional logs cannot show where latency bottlenecks or failures originate. OpenTelemetry tracks the end-to-end path of each request.

How It Works: As a request enters the system, an initial *Trace ID* is generated. As the request passes between microservices, each service creates a *Span* (recording start time, end time, and metadata) and propagates the Trace ID via HTTP headers (W3C Trace Context). Spans are exported to storage backends (such as AWS X-Ray or Jaeger).

Real-World Cloud Example: Pinpointing that a slow 3-second checkout request was caused by a specific 2.8-second SQL database query in the inventory microservice.

Related Concepts: Trace ID, Span, Context Propagation, OTel Collector, Jaeger, AWS X-Ray.

Concept: Grafana (Visualization & Dashboarding)

Beginner-Friendly Definition: Grafana is an open-source analytics and interactive visualization web application that allows users to query, visualize, alert on, and understand metrics, logs, and traces through unified dashboards.

Why It Exists: Raw numbers and log lines are difficult for humans to parse quickly during high-stress production incidents. Grafana turns telemetry data into intuitive visual charts, graphs, and heatmaps.

How It Works: Grafana connects to multiple data sources (such as Prometheus, Amazon CloudWatch, Elasticsearch, and Loki) without replicating data. Users write queries to build customizable dashboards and configure alerting rules.

Real-World Cloud Example: An SRE team monitoring a unified dashboard showing Kubernetes cluster node memory, container restart counts, and Application Load Balancer request rates in real time.

Related Concepts: Data Source, Time-Series Panel, Alerting Rules, Dashboard Variables, Amazon Managed Grafana.

3. The Four Cs of Cloud-Native Security

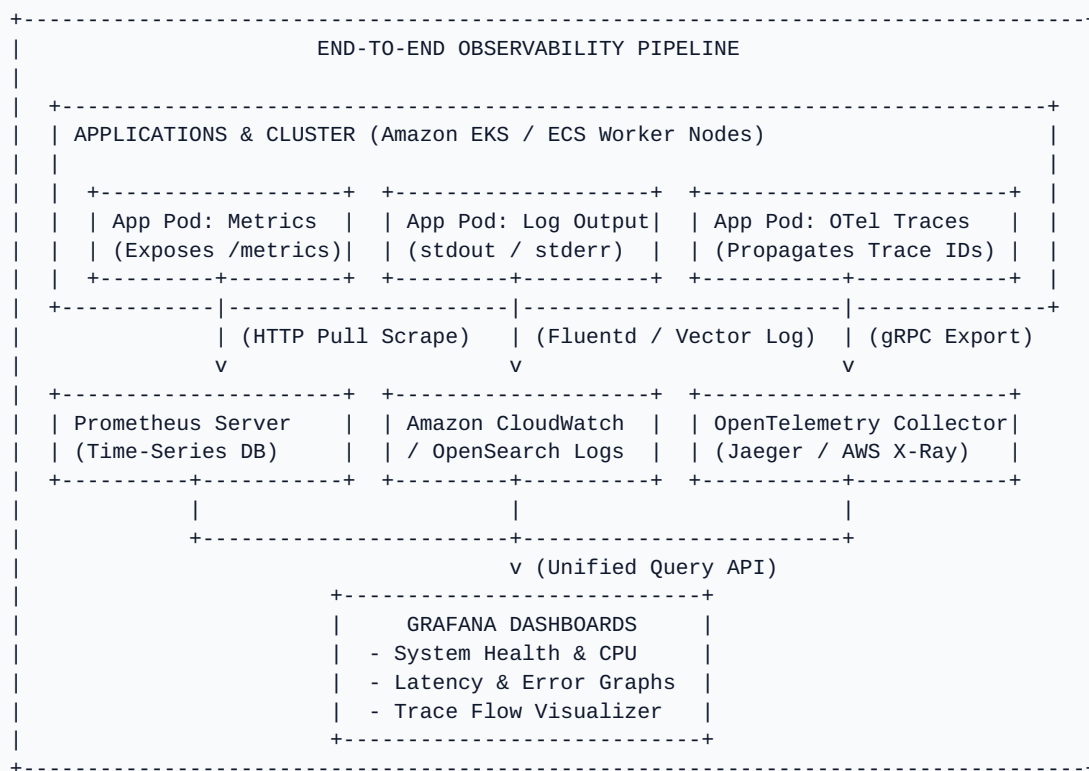
Cloud-native security is structured in concentric layers known as the **4Cs of Cloud-Native Security**: Cloud, Cluster, Container, and Code. Security at each outer layer directly protects the inner layers:

Layer	Security Focus & Responsibility	Enforcement Mechanism	Real-World AWS / K8s Example
1. Cloud	Infrastructure security, VPC isolation, and cloud account access.	AWS IAM policies, Security Groups, VPC Network ACLs.	Restricting EC2 worker nodes to private VPC subnets with no direct internet access.
2. Cluster	Kubernetes API access, resource authorization, and network segmentation.	RBAC, NetworkPolicies, Validating Admission Webhooks.	Blocking cross-namespace Pod communication using Kubernetes NetworkPolicies.
3. Container	Runtime security, base image vulnerabilities, and privilege limitation.	Non-root users, vulnerability scanners (Trivy), Seccomp profiles.	Scanning Amazon ECR container images for CVE vulnerabilities before deployment.
4. Code	Application logic, dependency security, and secret handling.	Static Application Security Testing (SAST), dependency auditing.	Fetching database credentials dynamically from AWS Secrets Manager instead of hardcoding.

4. The Three Pillars of Cloud Observability

Modern Site Reliability Engineering (SRE) relies on three complementary telemetry data types known as the **Three Pillars of Observability**:

- Metrics (Numeric Aggregations):** Quantifiable time-series values showing system performance over intervals (e.g., `cpu_usage_percent`, `http_requests_total`). Metrics are cheap to store, fast to query, and ideal for triggering automated alarms.
- Logs (Discrete Event Records):** Text-based timestamped records emitted when specific application events occur (e.g., "User 42 logged in successfully" or "Database connection timeout"). Logs provide rich contextual details for debugging specific failures.
- Traces (Request Lifecycles):** Structured records mapping the journey of a single user transaction across distributed microservices. Traces reveal inter-service dependencies, network transit times, and execution bottlenecks.



5. Secret Management & Zero-Trust Credentials

Hardcoding database passwords, API tokens, or encryption keys in source code or Dockerfiles is a primary cause of cloud security breaches.

Modern cloud environments implement **Dynamic Secret Injection**:

- **AWS Secrets Manager**: Stores encrypted secrets in AWS Key Management Service (AWS KMS) and automatically rotates database passwords on a scheduled basis without application downtime.
- **Kubernetes Secrets Store CSI Driver**: Mounts secrets directly from AWS Secrets Manager as read-only files into the container's filesystem in memory (tmpfs), ensuring secrets are never written to disk or stored in etcd unencrypted.
- **Ephemeral Credentials**: Applications use IAM Roles for Service Accounts (IRSA) to obtain short-lived, auto-expiring AWS security tokens, eliminating long-lived static API keys.

6. Practical Operational Troubleshooting

Resolving security and monitoring pipeline issues requires inspecting RBAC authorizations, scrape targets, and trace context propagation.

Failure Symptom	Root Cause	Diagnostic Action	Remediation Action
User / Pod 403 Forbidden on K8s API	Kubernetes RBAC Role or RoleBinding is missing the required API verb or resource permission.	<code>kubectl auth can-i <verb> <resource> --as=<user></code> or inspect ServiceAccount bindings.	Create or update the Kubernetes Role / ClusterRole with the appropriate permissions and re-bind.
Prometheus Target State DOWN	Prometheus scraper cannot reach application /metrics endpoint due to network policy or wrong port.	Check Prometheus UI under Status → Targets and verify Pod port configurations.	Update Kubernetes Service annotations or NetworkPolicies to allow inbound scraping on the metrics port.
Broken / Disconnected Traces	Intermediate microservice did not propagate the HTTP traceparent header to downstream calls.	Inspect outgoing HTTP request headers from the intermediate application code.	Ensure application uses OpenTelemetry HTTP client instrumentation to forward context headers automatically.
Grafana Dashboard Shows "No Data"	PromQL query syntax error, metric label mismatch, or Prometheus datasource connection failure.	Test the PromQL query directly in Prometheus Expression Browser to verify metric existence.	Correct metric name and label filters (e.g., verifying namespace="prod" vs namespace="production").

7. Explicit Architectural Relationships & Dependencies

This section explicitly documents the structural dependencies and multi-hop interactions between security and observability components for Graph RAG processing:

- **Kubernetes API server enforces RBAC:** The kube-apiserver validates incoming requests against Role and RoleBinding objects stored in etcd before allowing operations.
- **Prometheus scrapes metrics from Kubernetes Pods:** Prometheus sends periodic HTTP GET requests to container endpoints to collect time-series operational metrics.
- **OpenTelemetry instruments application microservices:** OTEL SDKs generate Spans and propagate Trace IDs across distributed services to track request latency.
- **Grafana visualizes Prometheus metrics:** Grafana executes PromQL queries against Prometheus datasources to render interactive health and latency dashboards.
- **Alertmanager sends alerts from Prometheus:** Alertmanager evaluates alert rules defined in Prometheus and dispatches notifications via Slack, PagerDuty, or email.
- **AWS Secrets Manager integrates with Amazon EKS:** The Secrets Store CSI Driver pulls encrypted secrets from AWS Secrets Manager and mounts them into Kubernetes Pods.
- **Amazon CloudWatch collects logs from Amazon ECS:** The AWS container agent forwards container stdout and stderr log streams directly to CloudWatch Log Groups.
- **Prometheus monitors Amazon EKS cluster health:** Prometheus scrapes node and container metrics to ensure EKS workloads stay within CPU and memory limits.

8. Key Takeaways, Entities & Interview Questions

8.1 Key Takeaways

- Cloud security follows the 4Cs model (Cloud, Cluster, Container, Code), applying the principle of least privilege at every layer.
- Kubernetes RBAC controls who can perform actions on cluster API resources using Roles and RoleBindings.
- Observability is built upon three pillars: Metrics (numerical aggregations via Prometheus), Logs (discrete event records), and Traces (request journeys via OpenTelemetry).

- Grafana provides a unified pane of glass to visualize metrics, logs, and traces across multi-cloud environments.
- Secrets should never be stored in plaintext or baked into container images; they must be managed dynamically using services like AWS Secrets Manager.

8.2 Important Entities & Classification

- **Security & Authorization:** Role-Based Access Control (RBAC), Role, ClusterRole, RoleBinding, ServiceAccount, Least Privilege, 4Cs Model.
- **Secret Management:** AWS Secrets Manager, AWS KMS, Kubernetes Secrets, Secrets Store CSI Driver.
- **Metrics & Monitoring:** Prometheus, PromQL, Scrape Target, Time-Series Database (TSDB), Exporter, Alertmanager.
- **Distributed Tracing:** OpenTelemetry (OTel), Trace ID, Span, Context Propagation, OTel Collector, Jaeger, AWS X-Ray.
- **Visualization & Dashboards:** Grafana, Data Source, Dashboard, Panels, Amazon Managed Grafana.
- **Log Aggregation:** Amazon CloudWatch, Fluentd, Vector, OpenSearch, Log Group.

8.3 Common Technical & Interview Questions

1. What is the difference between a Kubernetes Role and a ClusterRole?

Answer: A Role is namespace-scoped and grants permissions only within a single namespace (e.g., reading Pods in staging). A ClusterRole is cluster-scoped and grants permissions across all namespaces or on non-namespaced resources (e.g., Nodes, PersistentVolumes, and cluster-wide Pod monitoring).

2. How does Prometheus collect metrics from microservices, and why is the pull model preferred?

Answer: Prometheus uses a pull model where it periodically sends HTTP GET requests to configured `/metrics` endpoints on target containers. The pull model allows Prometheus to detect failed or unresponsive targets immediately (Target DOWN) and prevents crashing monitoring backends with sudden push traffic spikes.

3. What is distributed tracing, and how does OpenTelemetry propagate context across microservices?

Answer: Distributed tracing tracks the lifecycle of a request as it traverses multiple microservices. When a request begins, a unique Trace ID is generated. OpenTelemetry injects this Trace ID into outgoing HTTP headers (e.g., `traceparent`) and extracts it at downstream services, linking individual microservice operations (Spans) into a single unified trace tree.

4. Why are the Three Pillars of Observability (Metrics, Logs, Traces) complementary rather than interchangeable?

Answer: Metrics alert you that a problem is occurring (e.g., error rate spiked to 5%). Traces show you where the problem is located within the microservice call chain (e.g., payment service database call). Logs provide the exact details and stack trace explaining why the failure occurred.

5. How should application secrets be securely provided to containers running in production?

Answer: Secrets should be stored in an encrypted vault like AWS Secrets Manager. Using tools like the Kubernetes Secrets Store CSI Driver, secrets are retrieved dynamically at runtime and mounted as temporary memory-backed files (tmpfs) inside the container, preventing exposure in source control, build layers, or persistent disk storage.